Laboratorio di Machine Learning

Lorenzo Ciabatti

7196646

1 Obiettivo del laboratorio

L'obiettivo del presente lavoro è la riproduzione e l'analisi critica di un articolo scientifico recente inerente la predizione del tempo di consegna del cibo a domicilio:
`"Food Delivery Time Prediction Using Machine Learning Algorithms: A Comparative Study with Hybrid XGBoost-LSTM Architecture"`[2].
Il codice sorgente e il notebook sono disponibili su GitHub.
Il paper di riferimento confronta sei modelli di Machine Learning (Linear Regression, Decision Tree, Random Forest, Gradient Boosting, XGBoost, LSTM) e propone un'architettura ibrida originale, denominata **Hybrid XGBoost-LSTM**. In tale architettura, una rete LSTM elabora le sequenze storiche delle ultime cinque consegne di ciascun fattorino per produrre un embedding temporale; quest'ultimo viene successivamente concatenato alle feature statiche e fornito in input a un modello XGBoost.

Da un punto di vista intuitivo, i modelli confrontati rappresentano approcci concettualmente distinti al problema: la Regressione Lineare assume una relazione additiva tra le feature e il target; gli alberi di decisione e i loro ensemble (Random Forest, XGBoost) apprendono partizioni ricorsive dello spazio delle feature, particolarmente efficaci su dati tabulari eterogenei; XGBoost, in particolare, costruisce alberi in sequenza, ciascuno dei quali corregge gli errori residui del precedente. La LSTM, al contrario, è una rete neurale ricorrente dotata di memoria interna, pensata per catturare dipendenze in sequenze ordinate temporalmente — nel caso specifico, le abitudini di consegna recenti di ciascun fattorino. L'architettura ibrida mira a combinare la capacità di XGBoost di modellare relazioni non lineari su dati tabulari con la capacità della LSTM di estrarre pattern dalla storia recente di ciascun fattorino.

L'implementazione è stata condotta integralmente su Google Colab in ambiente Python, adottando un approccio incrementale orientato alla validazione step-by-step della pipeline.

2 Valutazione preliminare della qualità del paper

L'analisi critica preliminare del documento ha fatto emergere delle incongruenze testuali e metodologiche che hanno guidato le successive fasi di riproduzione:

- **Incongruenze nello split dei dati**: Il paper menziona inzialmente una suddivisione 70% training, 15% validation, 15% test, ma successivamente dichiara uno split 80% / 20%, supportato quest'ultimo dalle cifre assolute riportate (36.474 e 9.119 record).
- **Risultati mancanti e discrepanze tabellari**: Nonostante l'abstract e il testo descrivano le metriche eccellenti del modello ibrido (RMSE=4.32 min, MAE=3.41 min, R^2=0.912), la tabella riassuntiva dei risultati (Tabella V) si ferma all'analisi di XGBoost, omettendo totalmente sia l'architettura ibrida sia la LSTM standalone.

Nonostante tali criticità formali, il nucleo sperimentale è risultato solido e il dataset descritto (45.593 record, 19 feature grezze) è stato univocamente identificato nel dataset pubblico Kaggle `"Food Delivery Dataset"`[1].

3 Preparazione e pulizia dei dati

3.1 Ispezione e conversione dei tipi

Il dataset analizzato conta 45.593 osservazioni e 20 colonne. Un'ispezione preliminare ha rivelato l'assenza di valori 'NaN' nativi, imputabile alla presenza di stringhe letterali all'interno di colonne numeriche. Parallelamente, la variabile target `Time_taken(min)` è stata sottoposta a parsing tramite espressioni regolari per estrarne il valore numerico dalla formattazione originale (il formato nel dataset era: `"(min) 26"`).

3.2 Imputazione dei dati mancanti

A seguito della corretta conversione dei tipi, è emersa una percentuale di valori mancanti prossima all'1% (inferiore al 3,2% dichiarato dagli autori). Coerentemente con la metodologia originale, i valori mancanti continui sono stati sostituiti con la mediana, mentre quelli categorici con la moda. Le 1.731 righe prive dell'orario di ordinazione (`Time_Orderd`) sono state rimosse in quanto impossibili da imputare deterministicamente, restituendo un dataset pulito di 43.862 record.

Si segnala inoltre che, contrariamente a quanto dichiarato nel paper (rimozione di 187 record privi di target e di 312 osservazioni con tempo di consegna superiore a 60 minuti), nel dataset utilizzato il target `Time_taken(min)` non presentava valori mancanti dopo il parsing, e il valore massimo osservato era pari a 54 minuti: non è stato quindi necessario alcun filtraggio per outlier, poiché nessuna osservazione superava la soglia indicata dagli autori. Questa discrepanza ha successivamente trovato riscontro nell'osservazione che gli errori assoluti dei nostri modelli risultano sistematicamente inferiori a quelli dichiarati nel paper, suggerendo che la versione del dataset impiegata dagli autori possa differire leggermente, in termini di range del target, da quella pubblicamente disponibile su Kaggle.

3.3 Feature Engineering temporale e spaziale

A partire dai timestamp disponibili sono state ingegnerizzate le feature temporali proposte nel paper (`Day_of_Week`, `Month`, `Is_Weekend`, `Hour_of_Day`, `Meal_Period`). La variabile `Preparation_Time` ha mostrato una distribuzione discreta con passi di 5 minuti (5, 10, 15), suggerendo un arrotondamento a monte nel sistema di raccolta dati.

Per il calcolo della distanza geospaziale (Haversine distance) è stato necessario correggere bug del dataset documentato che presentava latitudini negative per l'India, risolto applicando il valore assoluto alle coordinate. L'estrazione dell'angolo di direzione (`Bearing_angle`) ha rivelato un'anomalia: i valori risultano confinati in un range ristretto (40,57° - 45,00°), suggerendo che le coordinate di consegna siano state parzialmente generate in modo sintetico, rendendo la feature poco informativa, contrariamente alle aspettative teoriche riportate nel testo.

4 Suddivisione dei dati e scaling

Per via dell'incoerenza documentale sullo split, si è optato per la suddivisione più supportata dai dati assoluti nel paper: 80% training e 20% test, stratificando per `City` e `Road_traffic_density`. Dal training set è stato estratto un ulteriore 12,5% (10% del totale) adibito a validation set. Lo scaling (`StandardScaler`) è stato fittato unicamente sul set di training e applicato ai soli modelli sensibili alla scala (Regressione Lineare e reti neurali) al fine di scongiurare fenomeni di data leakage.

5 Implementazione dei modelli e diagnostica

5.1 Baseline e XGBoost

Sono stati addestrati i modelli classici (Regressione Lineare, Ridge, Lasso, Decision Tree, Random Forest) utilizzando gli iperparametri suggeriti. La ricerca degli iperparametri ottimali per XGBoost tramite `RandomizedSearchCV` ha confermato esattamente la configurazione indicata nel paper (`n_estimators=20`, `max_depth=7`), validando fortemente l'allineamento della nostra riproduzione con l'esperimento originale.

5.2 LSTM e Modello ibrido: correzione del disallineamento indici

La costruzione delle sequenze temporali ha richiesto l'estrazione delle ultime 5 consegne per ciascun fattorino (ottenendo 37.263 sequenze valide). Durante la prima fase di addestramento dell'architettura ibrida, si è registrato un grave calo prestazionale ($R^2 < 0$), sintomo di una corruzione informativa. L'analisi diagnostica ha isolato la causa in un bug nell'allineamento degli indici di *pandas*: l'operazione `reset_index` eseguita prematuramente sul dataset ordinato temporalmente causava un mismatch tra le sequenze storiche e le feature statiche della consegna target. Corretto l'ordine delle operazioni di indicizzazione, il modello ha ripreso a convergere regolarmente. Per la costruzione finale dell'ibrido, l'embedding a 64 dimensioni è stato estratto dal modello ricorrente e concatenato alle 42 feature statiche, generando il vettore di input a 106 dimensioni per XGBoost.

5.3 Diagnosi del comportamento della LSTM standalone

Anche successivamente alla correzione del bug di allineamento, il modello LSTM standalone ha mostrato un comportamento anomalo: l'errore assoluto medio si stabilizzava costantemente intorno a 7,5 minuti, un valore prossimo a $0{,}8 \times \sigma$, dove $\sigma \approx 9{,}38$ minuti è la deviazione standard del target. Tale coincidenza numerica suggerisce che il modello, anziché apprendere pattern predittivi utili, convergesse verso una strategia pressoché equivalente alla previsione costante della media del target.

Per verificare se il plateau fosse imputabile a una scelta sub-ottimale degli iperparametri, è stata condotta un'iterazione sull'architettura: il *learning rate* è stato ridotto da 0,001 a 0,0005 e il numero di layer ricorrenti è stato diminuito da due a uno (da `LSTM(128)`→`LSTM(64)` a un singolo `LSTM(64)`), nel tentativo di ridurre il rischio di instabilità nell'ottimizzazione e il numero di parametri da stimare. Nonostante tali modifiche, il comportamento è rimasto sostanzialmente invariato, con il training loss in costante discesa e il validation loss stabile o in leggero peggioramento già dalle prime epoche — evidenza che il problema non risiedeva negli iperparametri, bensì nel design dell'input fornito al modello.

La causa è stata infine individuata nella natura stessa della sequenza costruita: ogni sequenza contiene esclusivamente le feature delle *cinque consegne precedenti* del fattorino (distanza, traffico, condizioni meteo di *quelle* consegne), ma non include alcuna informazione sulla consegna *corrente* che si intende predire. Il modello si trova quindi a dover stimare il tempo di una consegna specifica senza conoscere la sua distanza, il suo livello di traffico o il suo orario — informazioni che, come osservato nei modelli basati su feature statiche, risultano le più predittive in assoluto. È dunque ragionevole concludere che il limite della LSTM standalone non sia di natura ottimizzativa, ma strutturale: la sola storia del fattorino, scollegata dalle caratteristiche della consegna da prevedere, offre un segnale informativo debole. Questa osservazione fornisce inoltre una chiave di lettura diretta per il risultato discusso nella sezione contenente i Risultati, secondo cui il modello Hybrid — che invece combina l'embedding temporale con le feature statiche della consegna corrente — ottiene un guadagno solo marginale rispetto a XGBoost standalone: la componente realmente predittiva risiede nelle feature statiche, non nella sequenza storica.

6 Analisi dei risultati

Le metriche finali registrate sui dati di test confermano la gerarchia prestazionale dei modelli descritta nel paper originale, ma mettono in forte dubbio l'effettiva utilità dell'architettura ibrida su questo specifico set di dati.

Le prestazioni sono state valutate tramite tre metriche standard per problemi di regressione: il *Mean Absolute Error* (MAE), che esprime l'errore medio assoluto in minuti; il *Root Mean Squared Error* (RMSE), anch'esso espresso in minuti ma più sensibile agli errori di grande entità in virtù dell'elevamento al quadrato; e il coefficiente di determinazione R^2, compreso tra 0 e 1 (teoricamente anche negativo per modelli peggiori della media), che quantifica la proporzione di varianza del target spiegata dal modello.

Tabella 1: Confronto delle prestazioni: misurazioni empiriche vs valori dichiarati

Modello	**MAE (nostro)**	**RMSE (nostro)**	**R^2 (nostro)**	**MAE (paper)**	**RMSE (paper)**
Linear Reg.	4,76	5,95	0,595	7,32	9,41
Decision Tree	3,19	4,00	0,817	5,84	7,62
Random Forest	3,28	4,06	0,811	4,21	5,83
XGBoost	3,10	3,83	0,832	3,65	5,12
LSTM	7,45	9,28	0,022	–	–
Hybrid	**3,07**	**3,83**	**0,834**	**3,41**	**4,32**

I nostri errori assoluti risultano sistematicamente inferiori a quelli del paper. Ciò è ascrivibile alla mancanza di code estreme nel target del nostro dataset (valore massimo 54 minuti), in assenza dei 312 outlier > 60 min dichiarati nel testo. Il dato più critico è tuttavia il guadagno marginale dell'Hybrid XGBoost-LSTM: a fronte del 13,8% di miglioramento dell'RMSE dichiarato, la nostra riproduzione mostra prestazioni pressoché identiche a XGBoost standalone. La LSTM, incapace da sola di estrarre un segnale temporale forte ($R^2 = 0{,}02$), apporta un contributo trascurabile come meta-learner.

Un'osservazione ulteriore riguarda il confronto tra Decision Tree e Random Forest: nella nostra riproduzione il singolo albero di decisione ottiene un R^2 leggermente superiore a quello della foresta casuale (0,817 contro 0,811), capovolgendo l'ordine riportato nel paper. Il margine è contenuto (0,6 punti percentuali di R^2) e può essere spiegato dalla combinazione di due fattori: da un lato, il Decision Tree con gli iperparametri indicati (`max_depth=12`, `min_samples_leaf=20`) risulta già ben regolarizzato e non soffre significativamente di overfitting; dall'altro, il campionamento casuale delle feature operato da Random Forest (`max_features='sqrt'`) su un totale di 42 feature, molte delle quali a bassa informatività (ad esempio `Bearing_angle`, quasi costante come discusso in Sezione 3.3), può condurre occasionalmente alla selezione di split sub-ottimali nei singoli alberi dell'ensemble.

Distribuzione variabile target

La Figura 1 mostra la distribuzione della variabile target sull'intero dataset pulito. La media empirica (26,3 minuti) risulta prossima ai 26,7 minuti dichiarati dagli autori, a conferma della coerenza tra il dataset utilizzato e quello originale. La distribuzione presenta tuttavia un andamento irregolare, con picchi e valli alternati non attribuibili al caso: tale pattern discende direttamente dalla natura discreta delle feature sottostanti, in particolare `Preparation_Time`, che nel dataset assume esclusivamente i valori 5, 10 e 15 minuti. La combinazione di poche variabili quantizzate genera così una distribuzione del target concentrata su specifiche combinazioni di valori piuttosto che su un continuo, un dettaglio non discusso nel paper originale ma rilevante per l'interpretazione dei risultati successivi.

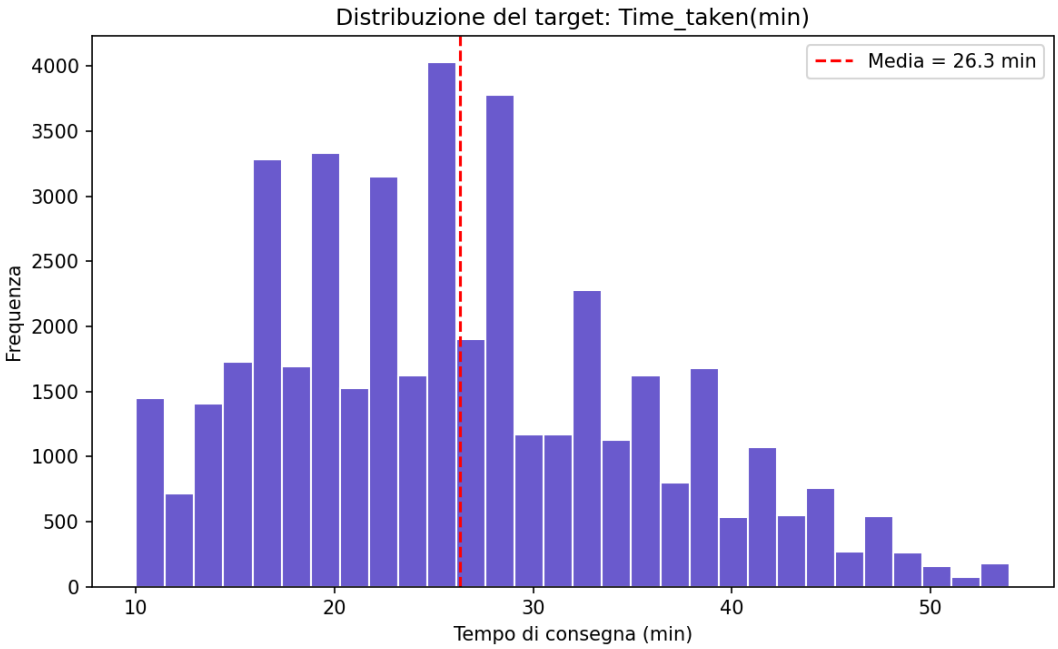


Figura 1: Distribuzione della variabile target `Time_taken(min)`. I picchi discreti confermano la natura a scatti delle feature sottostanti.

Confronto prestazioni modelli addestrati

La Figura 2 riassume le prestazioni di tutti i modelli addestrati, ordinati per R^2 decrescente. Il grafico rende immediatamente visibili tre raggruppamenti distinti: un primo gruppo (Hybrid XGBoost-LSTM, XGBoost, Decision Tree, Random Forest) con R^2 compreso tra 0,81 e 0,83 ed errori assoluti contenuti in un intervallo di 3-4 minuti; un secondo gruppo (Linear Regression, Ridge, Lasso), sostanzialmente equivalente tra loro, con R^2 intorno a 0,60 ed errori quasi doppi rispetto al primo gruppo; infine il modello LSTM standalone, che si stacca nettamente dagli altri con un crollo del R^2 fino a 0,02 e i valori di errore più elevati in assoluto (MAE 7,45 minuti). Il grafico evidenzia con chiarezza il divario prestazionale tra approcci basati su alberi/ensemble e la rete ricorrente pura, coerentemente con quanto discusso in precedenza a proposito del segnale temporale debole disponibile alla LSTM.

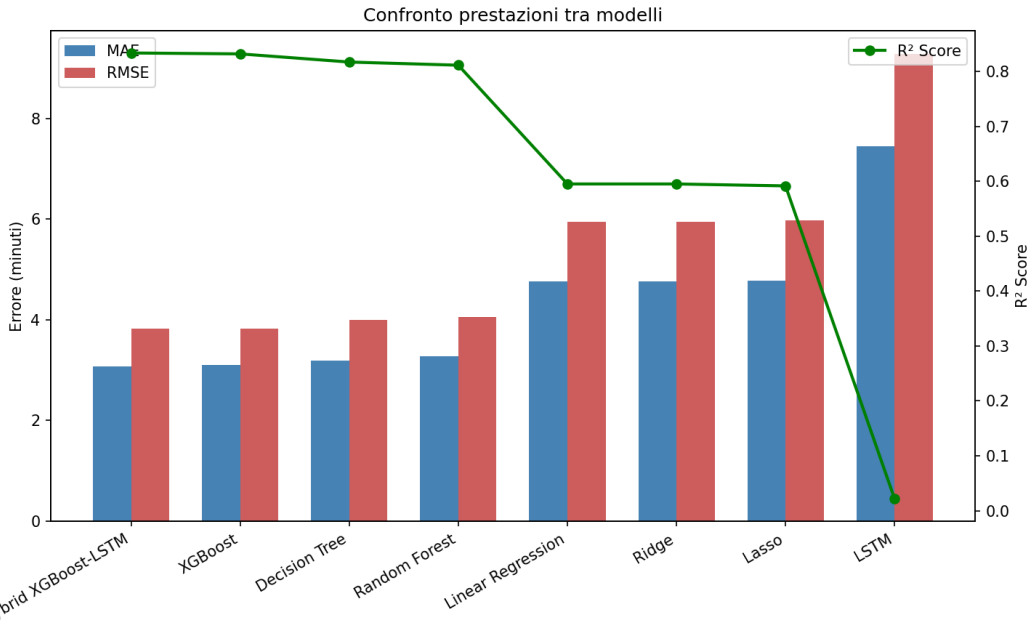


Figura 2: Confronto delle prestazioni dei modelli. Evidente il divario tra i modelli ad albero/ensemble e la debolezza del modello LSTM puro.

Confronto qualità predittiva modelli addestrati

La Figura 3 confronta visivamente la qualità predittiva di Linear Regression, XGBoost e del modello Hybrid tramite scatter plot Predicted vs Actual. Il primo pannello mostra una dispersione marcata attorno alla diagonale di riferimento, più accentuata per i valori estremi del target: questa cosa conferma empiricamente l'ipotesi, sollevata anche dagli autori, che la relazione tra le feature e il tempo di consegna non sia lineare. I pannelli relativi a XGBoost e al modello Hybrid mostrano invece nuvole di punti molto più addensate attorno alla diagonale e, soprattutto, risultano pressoché sovrapponibili tra loro: questa evidenza visiva conferma direttamente quanto osservato nelle metriche aggregate, ossia che l'aggiunta della componente LSTM non produce un miglioramento qualitativamente apprezzabile rispetto a XGBoost standalone. In tutti e tre i pannelli si osserva inoltre una disposizione dei punti "a colonne verticali", ulteriore manifestazione della natura discreta di `Preparation_Time` già discussa nella fase di feature engineering.

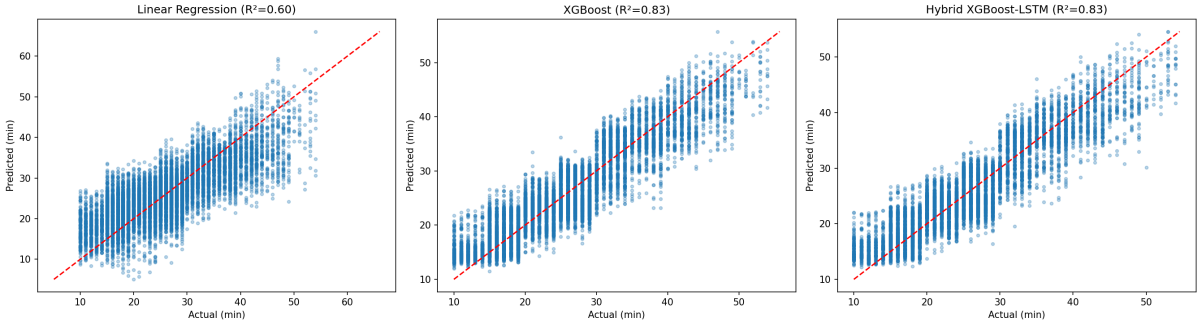


Figura 3: Scatter plot Predicted vs Actual per Regressione Lineare, XGBoost e modello Ibrido. XGBoost e l'Ibrido risultano visivamente sovrapponibili.

Andamento del training del modello LSTM

La Figura 4 documenta l'andamento del training del modello LSTM standalone, riportando l'errore assoluto medio su training e validation set per epoca. La curva di training decresce in modo monotono e marcato (da 9,47 a 6,46 minuti), mentre la curva di validation raggiunge il proprio minimo intorno all'epoca 6-7 (circa 7,68 minuti) per poi risalire progressivamente fino a 8,35 minuti, delineando un chiaro fenomeno di overfitting a partire dall'epoca 7. Il meccanismo di early stopping (`patience=10` epoche, `restore_best_weights=True`) ha correttamente interrotto l'addestramento e ripristinato i pesi del punto di minimo in validazione, evitando un ulteriore degrado delle prestazioni sul test set. Questa curva costituisce l'evidenza diagnostica più diretta a supporto della conclusione che il segnale temporale disponibile alla LSTM, in questo dataset, sia intrinsecamente debole: il modello non riesce a superare un plateau di errore prossimo a quello di una predizione costante pari alla media del target.

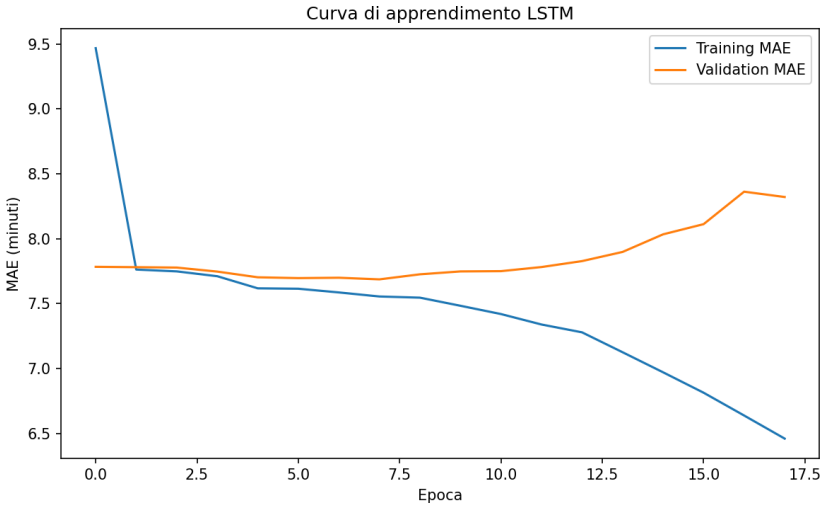


Figura 4: Curva di apprendimento del modello LSTM puro. Si nota la divergenza tra training e validation loss a partire dall'epoca 7.

7 Limiti e Conclusioni

Tra i limiti della riproduzione si segnalano l'impossibilità di ricreare l'esatta percentuale di valori mancanti (1% vs 3,2%) e il numero finale di feature codificate (42 vs 34), dovuti alla mancanza di un repository del codice sorgente degli autori.

Il lavoro svolto ha comunque replicato fedelmente la metodologia, affrontando e superando problematiche complesse nella gestione dei dati e nell'ingegnerizzazione del software (come il bug di indicizzazione). La conclusione scientifica fondamentale è che **l'apporto della memoria a breve termine (LSTM) alla predizione dei tempi di consegna, su questo specifico dataset, è minimo e non giustifica il costo computazionale dell'architettura ibrida**, smentendo parzialmente l' incremento di performance ($R^2 > 0.90$) rivendicato nell'articolo originale. Tale discrepanza ribadisce l'importanza cruciale della riproducibilità indipendente nella ricerca applicata al Machine Learning.

Dichiarazione sull'utilizzo di strumenti di Intelligenza Artificiale

Durante lo svolgimento del presente laboratorio, è stato utilizzato il supporto di modelli linguistici come assistenti alla programmazione. L'utilizzo di tali strumenti è stato strettamente limitato al supporto al debugging (ad esempio, per agevolare l'individuazione del disallineamento degli indici nella libreria *pandas*) e all'ottimizzazione e organizzazione formale del codice all'interno dell'ambiente Google Colab.

Tutte le scelte metodologiche, l'analisi critica del paper originale, l'interpretazione dei risultati e la stesura della presente relazione sono frutto di un lavoro individuale, inoltre sono stati costantemente supervisionati e validati ogni output prodotto dall'intelligenza artificiale.

Riferimenti bibliografici

[1] Gaurav Malik. Food delivery dataset. Dataset online.

[2] Leelawati Pokhrel, Mohd Arsalan, Babita Tiwari, and Puneet Garg. Food delivery time prediction using machine learning algorithms: A comparative study with hybrid xgboost-lstm architecture. In *2026 6th International Conference on Emerging VLSI and Semiconductor Technology for AI and Computing Applications (EVST)*, pages 1–7, 2026.